Event-Driven Image Annotation and Retrieval System

Osama Alshaykh

© 2026 Osama Alshaykh — Boston University EC530
Shared for educational use with attribution

EC530 Project 2

Event-Driven Image Annotation and Retrieval System

Two-week masters-level assignment
Pub-sub + documentDB + vector DB
Slides were made or assisted using ChatGPT
Osama Alshaykh

Learn architecture, not just models

Testability is required

Students design a modular image-processing pipeline that receives image events, stores variable annotations as documents, and adds embedding-based similarity search.

Highlights

- This project combines modular systems, pub-sub, document databases, and vector search in one coherent system.
- The workflow is naturally asynchronous: submission, inference, storage, indexing, and retrieval do not need one blocking call
- As you recall, annotations are variable and nested, so students must justify a document model instead of forcing fixed tables.
- Embeddings add a new capability: similarity search over images or objects, not just metadata lookup.
- An event generator makes unit testing concrete through contracts, mocking, deterministic replay, and fault injection.

You are graded on architecture, testability, and design justification — not on training a model.

Use cases

- Mission: Visual Object retrieval system. Search using natural language and get the objects
- Use cases
  a. User can upload images. The run via an AI system, it detects objects and provides embedding for those objects.
  b. User searches for something and retrieves objects that match

For this project, PLEASE dont focus on AI. Instead of AI, define the module, APIs, messages it supports and simulate the component

Query

- I give you an image, give me the top k similar images
- I give you a topic, find the images that match or close to that topic

You will not implement matching algorithms. We will use vector management systems

User flows

Upload images

image.submitted *who should listen to it?*
inference worker *when it is is done what should it do?*

inference.completed *who should isten to it?*
annotation service *when it is is done what should it do?*

annotation.stored *who should listen to it?*

embedding.created *who should listen to it?*

Search

Query submitted *who should generates it and who listens to it?*

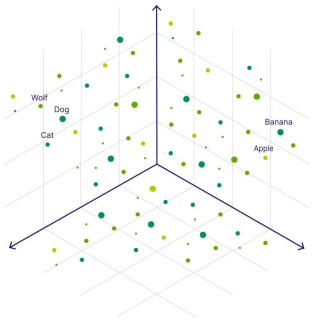
query system worker *what should it do, generate message and who should listen to that message?*

Query.completed

CLI service

Is there order to the messages?

Brief Introduction to Vector Databases
We will go over details next week



Simple definition

A vector database stores embeddings and supports nearest-neighbor search so we can retrieve items that are similar in meaning or visual content, not just identical in keywords.

embedding model

distance metric

top-k retrieval

Real Life Examples

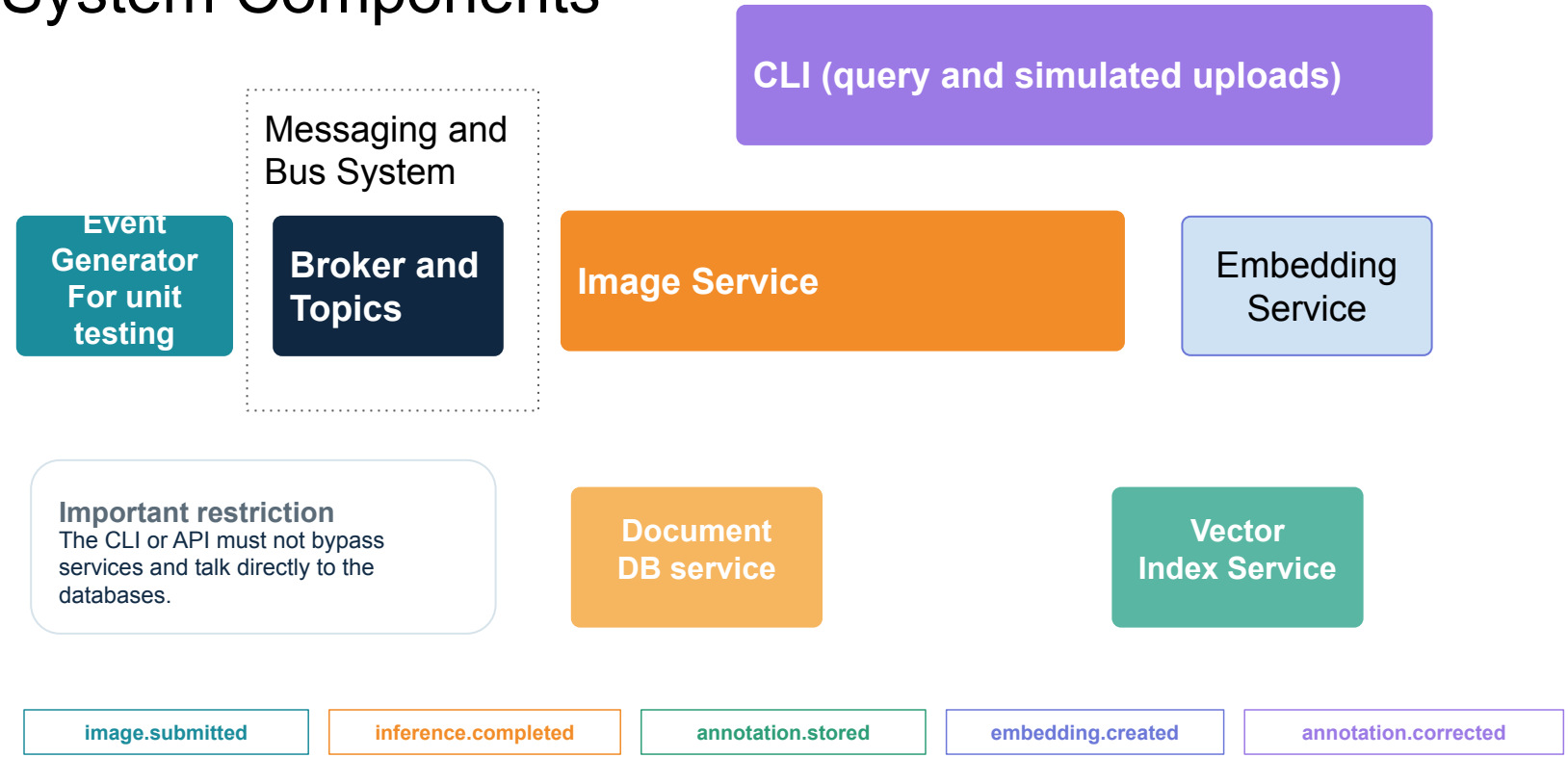
- Geo locations.
  - Can you find all places close to this address?
  - Can you find a near bu hospital?
- Recommendations
  - Who has expertise in this area?
  - Can you find all messages on quality of EC530 as a class?
- Image Search:
  - I want to see my cat with their halloween costume?

Vector Search

Our objective is to learn thre system and not just use them

- We will build a system that uses FAISS as a vector index
- We will discuss this next Tuesday in class
- We will also discuss Vector Database Management Systems (VDBMS). It will be a bonus to use such systems and learn them.

System Components



Event Generator

System Guarantees

dempotency
- Duplicate events do not create duplicate state

Robustness
- Malformed events do not crash the system

Eventual Consistency
- System may be temporarily incomplete but converges

Accurate Queries
- Queries reflect the current processed state

You need to devise unit tests that reflect these guarantees

Failure modes to inject: duplicates, dropped messages, delayed delivery, and subscriber downtime.

Do Definsive Testing

Example event schema

```

  "type"  "publish"
  "topic"  "image.submitted"
  "event_id"  "evt_1042"
  "payload"
    "image_id"  "img_1042"
    "path"  "images/street_1042.jpg"
    "source"  "camera_A"
    "timestamp"  "2026-04-07T14:33:00Z"


```

Deterministic mode with seed

Replay mode from sample dataset

First Exercise today

- Define the services and who owns the data. Only one service should only a data store or table. Other services should communicate via events. Suggestions
  - Upload Service
  - Inference Service
  - Document DB Service
  - Embedding Service
  - CLI Service
- Define what each service is responsible for and owns
- Define Topics
- Define who publishes and subscribes

Define your messaging system for your pub-sub

- What are the topics
- Who subscribe to which topics
- What is the message structure
- Create unit tests for it, e.g.,
  - A valid event includes topic, event_id, payload, and timestamp.
  - The generator can be tested without a live broker by mocking publish().
  - Duplicate events do not create duplicate database state if idempotency is implemented correctly.
  - Malformed or late events are rejected or logged without crashing the system.
- I recommend using REDIS pub-sub. You can use any messaging and bus systems.

Example event schema Example

```
1
2   "type"   "publish"
3   "topic"   "image.submitted"
4   "event_id"   "evt_1042"
5   "payload"
6     "image_id"   "img_1042"
7     "path"   "images/street_1042.jpg"
8     "source"   "camera_A"
9     "timestamp"   "2026-04-07T14:33:00Z"
10
11
```

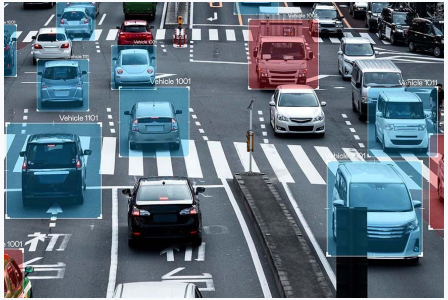
Deterministic mode with seed

Replay mode from sample dataset

Document Database

One image record in the document DB

```
{
  "image_id": "img_1042",
  "camera": "A",
  "objects": [
    {"label": "car", "bbox": [12, 44, 183, 200], "conf": 0.93},
    {"label": "person", "bbox": [230, 51, 286, 210], "conf": 0.88}
  ],
  "review": {
    "status": "corrected",
    "notes": ["car -> truck on 2nd pass"]
  },
  "history": [
```



Why is this easier to evolve than a fixed relational schema?

Need to design and define

- Each image can contain a different number of objects.
- Objects contain nested fields such as labels, boxes, confidence, or reviewer notes.
- Corrections and model versions create evolving fields over time.
- A document database stores JSON-like records directly and avoids premature normalization.

Two-week project plan

Days 1-2
Define modules, topics, event schema, and repository structure.

Days 3-4
Implement event generator, broker interface, and unit tests with mocks.

Days 5-7
Store annotations in the document DB and wire inference/annotation flow.

Days 8-10
Add embeddings, vector search, and one query path through a service layer.

Days 11-14
Refine tests, add failure handling, prepare README and short architecture video.

Instructor checkpoints: architecture review in week 1; testing and design justification review in week 2.

Focus of this week

- By Tuesday April 14th, you need to finish
  - Project structure
  - Event and message definitions for each module
  - Use REDIS (Topics and messages)
  - Complete testing of the system and unit testing of all messages
- This means your focus is on using and implementing the messaging system on REDIS and testing it very week.
- To make our two week deadline, please be ready by Tuesday April 14th with the deliverables

Next week

- I will provide you with simulated data to use for annotations and embeddings
- We will implement the services

What you are not building

- ❌ Do NOT train models
- ❌ Do NOT implement ANN algorithms
- ✅ Use provided embeddings OR simple model (will be provided next week)
- ✅ Focus on system integration

Thank you